

BOB-137 " merge service (7187) wedge: captured forensics

Revision: 2 Last modified: 2026-08-20T15:00:00Z

Captured 2026-08-20 on the live stack while the defect was ACTIVE.
Evidence class: runtime (Â§11.4.226) " live process, socket and thread state, not source inspection.

1. The wedge, measured

Container qbittorrent-proxy, up 3h53m, reported Up 4 hours (healthy):

```
curl --max-time 6 http://localhost:7186/      -> HTTP 200 in 0.096s
curl --max-time 6 http://localhost:7187/      -> HTTP 000 after 6.004s
```

Both ports are served by the SAME process " ss -ltnp shows pid 2330069 holding fd 3 (7186) and fd 7 (7187). So this is not a crashed worker: one loop inside a live process stopped servicing requests while another in the same process kept working.

2. Thread census (the GIL-starvation signature)

16 threads. One in state R with wchan=0; one in do_sys_poll; the other 14 all in futex_wait_queue. Process CPU 20.6% while nominally idle.

A thread busy-looping in Python holds the GIL, and every other thread queues on the GIL futex. 7186 survives because do_sys_poll releases the GIL; the 7187 async loop needs sustained GIL time to service a request and starves.

See threads.txt.

3. Socket state

Seven sockets held by the process in CLOSE-WAIT with unread bytes still queued (Recv-Q 162, 162, 162, 79, 1, 1, 1): clients sent a request and hung up, and the application neither read it nor closed the fd. At first observation the 7187 LISTEN socket also had an unaccepted backlog (Recv-Q 6).

NOT fd exhaustion " 48 of 16384 fds open. See sockets.txt.

4. Log silence

The container log's final entry predates the observation by ~2 hours; the service processed nothing in that window. See `container_log_tail.txt`.

5. Root cause: NOT ESTABLISHED

All four `while True` loops in the service (`routes.py:166`, `search.py:1169`, `streaming.py:161`, `streaming.py:359`) are correctly bounded and awaited, so the spin is not a naive unslept loop.

`py-spy` could not attach to name the spinning frame: the host runs `kernel.yama.pttrace_scope=1`, which denies non-child attach. Per the [Â§11.4.102 Iron Law](#) no fix is proposed until the spinning frame is identified.

6. UPDATE " the wedge is TRANSIENT and self-clearing

Roughly 15 minutes after the measurements above, and with NO restart and no intervention, 7187 began answering normally:

```
curl --max-time 8 http://localhost:7187/health -> HTTP 200 in 0.021s
```

This materially changes the diagnosis: the service is not permanently wedged, it stalls for a bounded-but-long period (≥ 2 h of log silence here) and then recovers. That is consistent with a long-running GIL-hogging operation completing, rather than a true deadlock.

HONEST BOUNDARY ([Â§11.4.6](#)): the CAUSE of the recovery is UNKNOWN. A `py-spy` attach was attempted (and failed with `EPERM`) shortly before recovery was observed; whether that attempt perturbed the process is UNCONFIRMED. The recovery is NOT claimed to be spontaneous, only observed.

Consequence for reproduction: a repro must SOAK " a point-in-time probe will often find the service healthy and conclude, falsely, that there is no defect ([Â§11.4.201\(6\) false-null](#)).

7. Discovery channel ([Â§11.4.238](#))

Found by an agent hand-probing the host during an unrelated investigation " NOT by automated QA. Coverage escape. The sibling item BOB-138 records the health check that was structurally incapable of observing this.